

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Полтавська політехніка
імені Юрія Кондратюка»

Кафедра комп'ютерних та інформаційних технологій і систем

Звіт
з лабораторної роботи №4
з дисципліни «Технології на платформі NET»,
тема: «Розробка вебсерверів засобами фреймворку ASP.NET»

Виконав:

Студент групи 404-ТН

Плаксюк Я.М.

Перевірив:

Викладач каф. КІТС Тютюнник П.Б.

Полтава, 2025

Завдання

1. Створити проєкт за шаблоном ASP.NET Core Web API {Назва тематики}.REST використовуючи Visual Studio або .NET SDK: `dotnet new webapi -n {Назва тематики}.REST`
2. Створити папку Models, у якій будуть зберігатися моделі, які будуть використовуватися контролерами у проєкті {Назва тематики}.REST.
3. Створіть контролери у папці Controllers, які будуть описувати REST API для виконання CRUD операцій (створення, читання, оновлення, видалення) над моделями у вашій тематичі, які ви зберігали у базі даних у третій лабораторній, при цьому моделі, що використовуються у контролерах, повинні бути описані у проєкті {Назва тематики}.REST, щоб відповідати 3-рівневій архітектурі. Наприклад, якщо у вас є сутність Bus, що зберігається у БД, ви створите новий клас BusModel у папці Models нового проєкту, який використовуватиметься у контролерах. Також зверніть увагу, щоб моделі у контролерах, мали лишень необхідні властивості, та якщо якісь властивості зайві, для якихось із методів - створіть нову модель без цих властивостей. Також у лабораторній роботі повинно бути не менше двох сутностей для яких реалізовані контролери.
4. При проєктуванні Web API зверніть увагу, щоб створений API відповідав 4-ій умові побудови REST-додатку по Філдингу - однорідності інтерфейсу (див. 31 слайд), тобто використовувалися унікальні назви сутностей, правильні HTTP методи та поверталися правильні HTTP коди у HTTP відповідях, наприклад 404 якщо сутність не знайдена, або 201 якщо нова сутність створена.
5. Використовуйте асинхронну версію дженерік CRUD сервісу, що використовує репозиторій для доступу до даних та який був реалізований у 3ій лабораторній роботі, у створених контролерах.:

```
public interface ICrudServiceAsync<T>
```

```
{
```

```
    public Task<bool> CreateAsync(T element);
```

```
        public Task<T> ReadAsync(Guid id);

        public Task<IEnumerable<T>> ReadAllAsync();

        pub

        lic Task<IEnumerable<T>> ReadAllAsync(int page, int amount);

        public Task<bool> UpdateAsync(T element);

        public Task<bool> RemoveAsync(T element);

        public Task<bool> SaveAsync();

    }
```

Імплементацію CRUD сервісу, репозиторіїв та контексту, додавайте у контролери використовуючи вбудовану у ASP.NET Core підтримку впровадження залежностей(Dependency Injection).

До PR готової лабораторної роботи додайте PDF файл у якому будуть результати виконання запитів, використовуючи створений REST API. Можете скористатися сторінкою Swagger, яка генерується ASP.NET застосунком, або застосунком Postman.

Виконання

Завдання 1

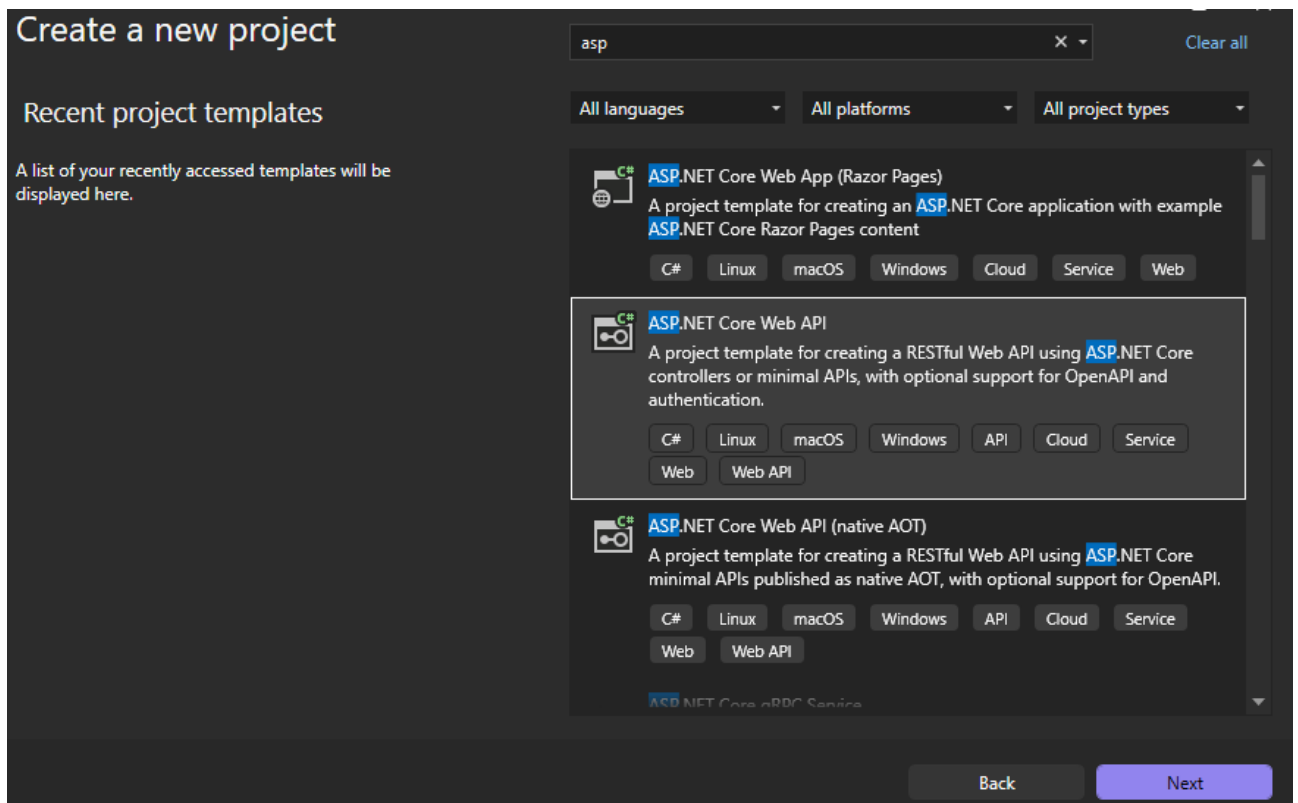


Рис. 1 – Створення проєкту за шаблоном ASP.NET

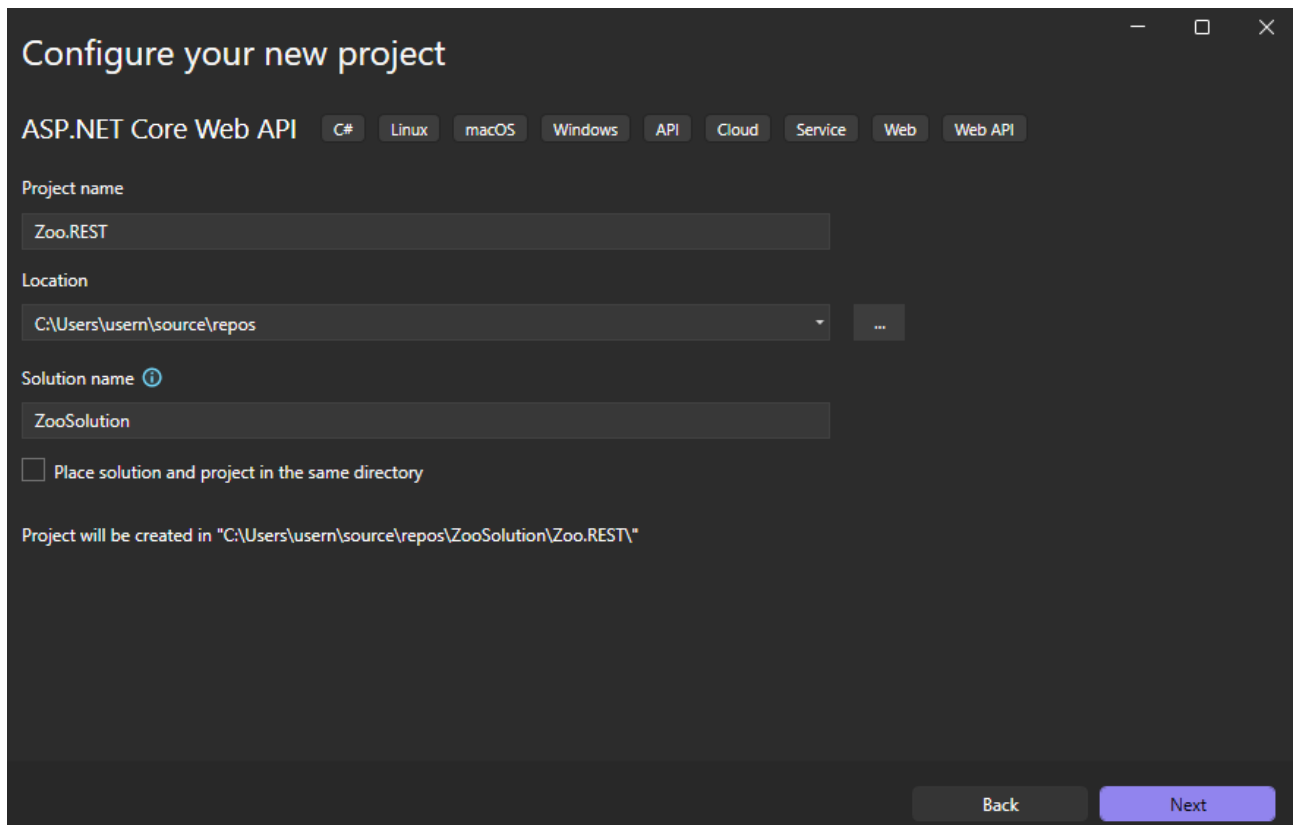


Рис. 2 – Назва мого проєкту за шаблоном ASP.NET

Завдання 2

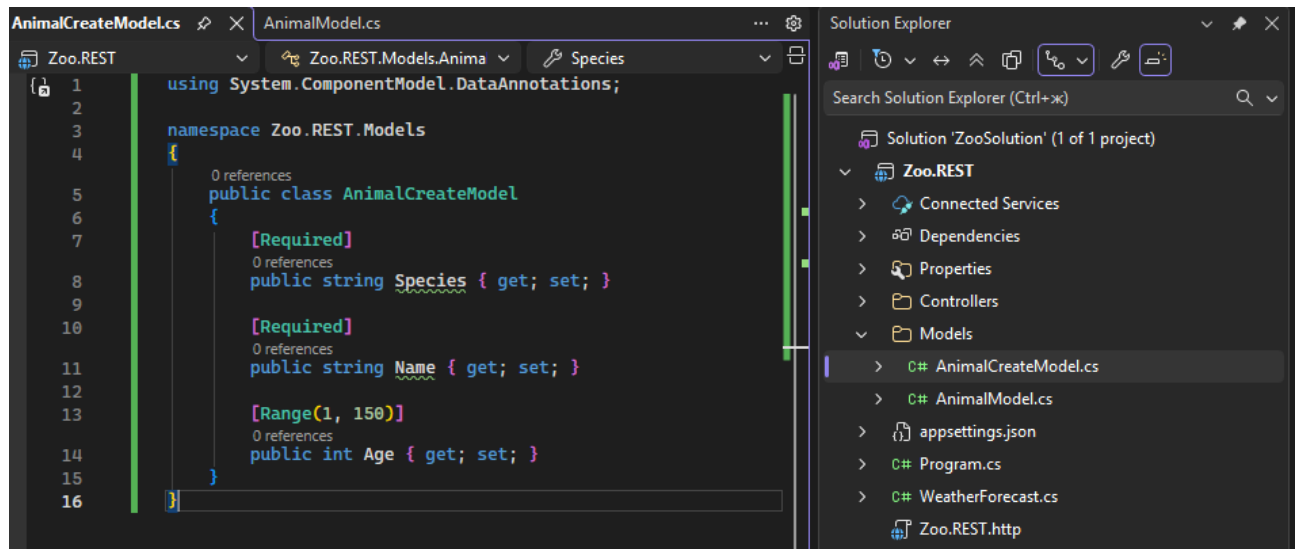


Рис. 3 – Доданий файл `AnimalCreateModel.cs`

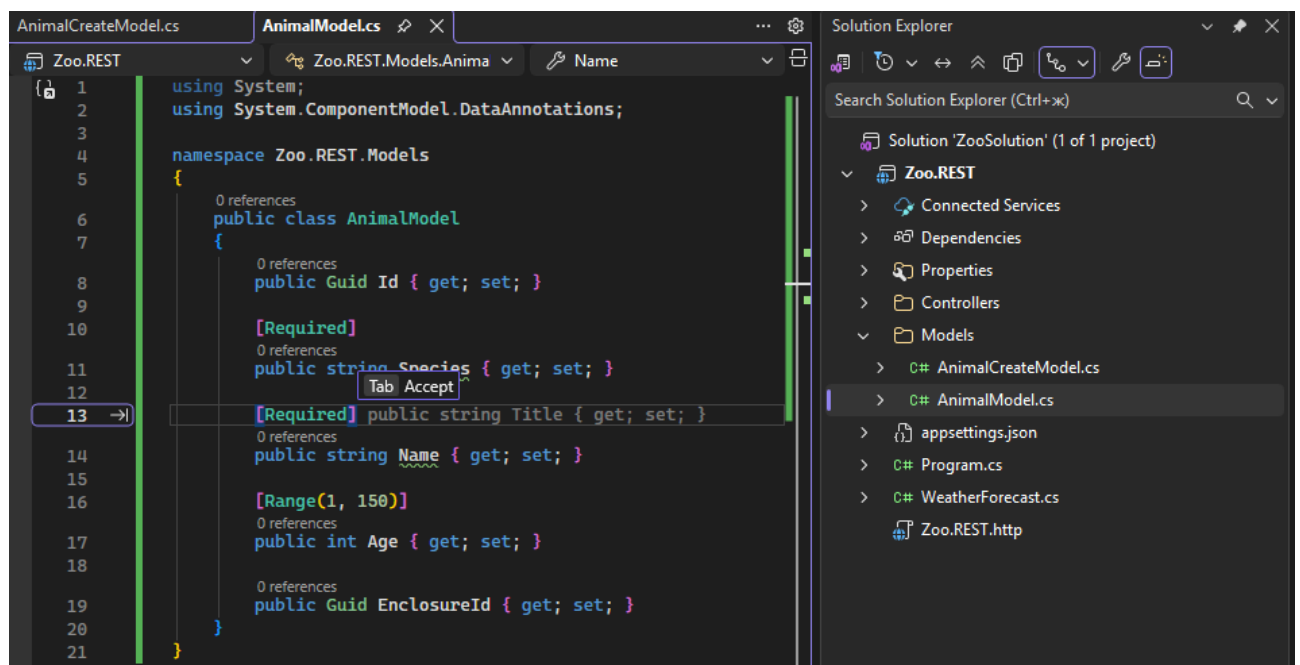
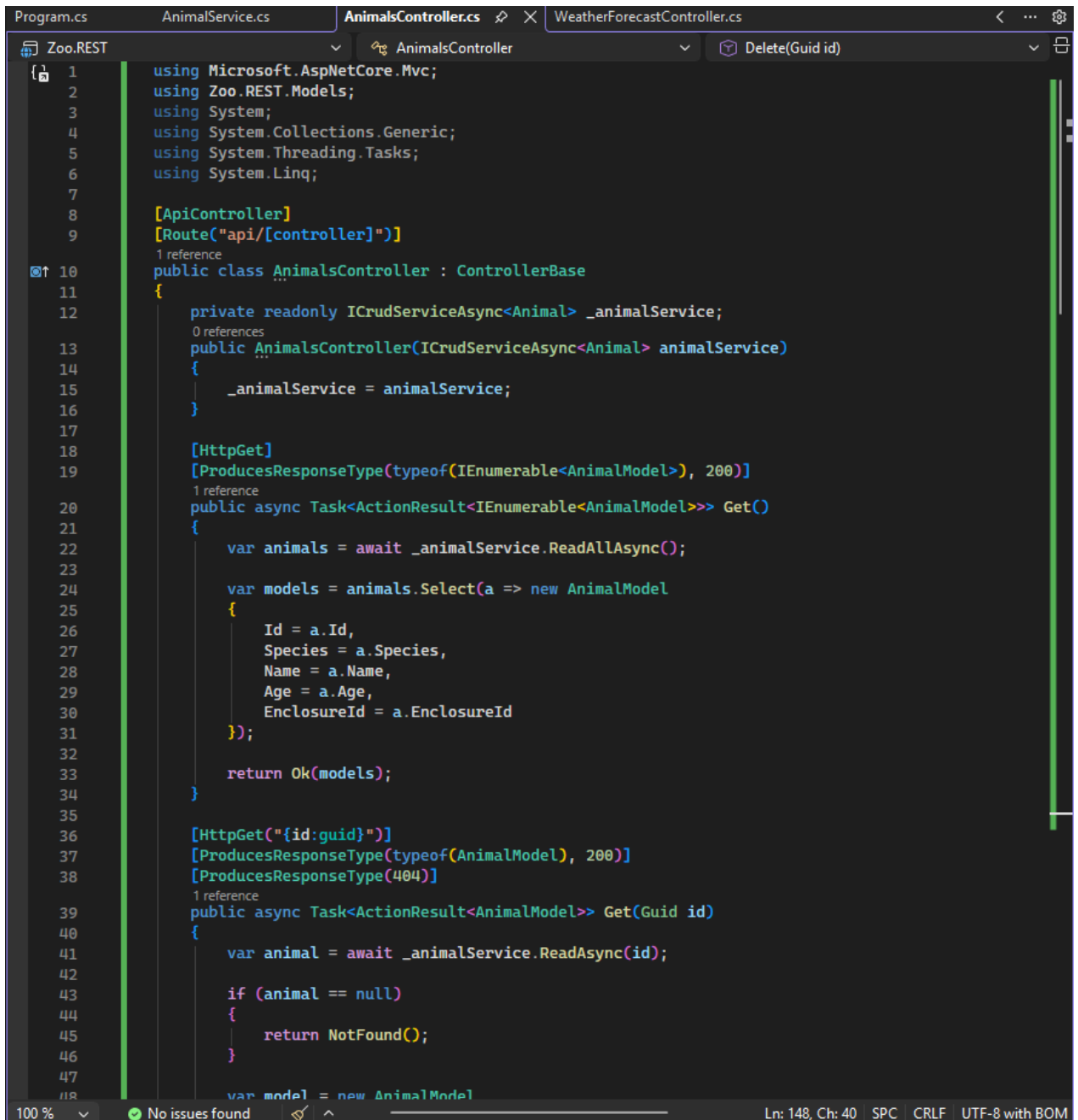


Рис. 4 – Доданий файл `AnimalModel.cs`

Завдання 3 та 4



```
Program.cs  AnimalService.cs  AnimalsController.cs  WeatherForecastController.cs
Zoo.REST
1  using Microsoft.AspNetCore.Mvc;
2  using Zoo.REST.Models;
3  using System;
4  using System.Collections.Generic;
5  using System.Threading.Tasks;
6  using System.Linq;
7
8  [ApiController]
9  [Route("api/[controller]")]
10 public class AnimalsController : ControllerBase
11 {
12     private readonly ICrudServiceAsync<Animal> _animalService;
13     public AnimalsController(ICrudServiceAsync<Animal> animalService)
14     {
15         _animalService = animalService;
16     }
17
18     [HttpGet]
19     [ProducesResponseType(typeof(IEnumerable<AnimalModel>), 200)]
20     public async Task<ActionResult<IEnumerable<AnimalModel>>> Get()
21     {
22         var animals = await _animalService.ReadAllAsync();
23
24         var models = animals.Select(a => new AnimalModel
25         {
26             Id = a.Id,
27             Species = a.Species,
28             Name = a.Name,
29             Age = a.Age,
30             EnclosureId = a.EnclosureId
31         });
32
33         return Ok(models);
34     }
35
36     [HttpGet("{id:guid}")]
37     [ProducesResponseType(typeof(AnimalModel), 200)]
38     [ProducesResponseType(404)]
39     public async Task<ActionResult<AnimalModel>> Get(Guid id)
40     {
41         var animal = await _animalService.ReadAsync(id);
42
43         if (animal == null)
44         {
45             return NotFound();
46         }
47
48         var model = new AnimalModel
49         {
50             Id = animal.Id,
51             Species = animal.Species,
52             Name = animal.Name,
53             Age = animal.Age,
54             EnclosureId = animal.EnclosureId
55         };
56         return Ok(model);
57     }
58 }
```

Рис. 5 – Доданий файл AnimalsController.cs частина 1

```
58 }
59
60 [HttpPost]
61 [ProducesResponseType(typeof(AnimalModel), 201)]
62 [ProducesResponseType(400)]
63 public async Task<ActionResult<AnimalModel>> Post([FromBody] AnimalCreateModel createModel)
64 {
65     if (!ModelState.IsValid)
66     {
67         return BadRequest(ModelState);
68     }
69
70     var newAnimal = new Animal
71     {
72         Species = createModel.Species,
73         Name = createModel.Name,
74         Age = createModel.Age
75     };
76
77     var success = await _animalService.CreateAsync(newAnimal);
78
79     if (success)
80     {
81         var createdModel = new AnimalModel
82         {
83             Id = newAnimal.Id,
84             Species = newAnimal.Species,
85             Name = newAnimal.Name,
86             Age = newAnimal.Age
87         };
88
89         return CreatedAtAction(nameof(Get), new { id = createdModel.Id }, createdModel);
90     }
91
92     return BadRequest("Помилка");
93 }
94
95 [HttpPut("{id:guid}")]
96 [ProducesResponseType(204)]
97 [ProducesResponseType(404)]
98 [ProducesResponseType(400)]
99 public async Task<IActionResult> Put(Guid id, [FromBody] AnimalModel updateModel)
100 {
101     if (id != updateModel.Id)
102     {
103         return BadRequest("Помилка ID");
104     }
105
106     var existingAnimal = await _animalService.ReadAsync(id);
```

Рис. 6 – Доданий файл AnimalsController.cs частина 2

Завдання 5

```
1 using System;
2 using System.Collections.Generic;
3 using System.Threading.Tasks;
4
5 public interface ICrudServiceAsync<T> where T : class
6 {
7     public Task<bool> CreateAsync(T element);
8     public Task<T> ReadAsync(Guid id);
9     public Task<IEnumerable<T>> ReadAllAsync();
10    public Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
11    public Task<bool> UpdateAsync(T element);
12    public Task<bool> RemoveAsync(T element);
13    public Task<bool> SaveAsync();
14 }
```

Рис. 7 – Доданий файл ICrudServiceAsync.cs

Завдання 6

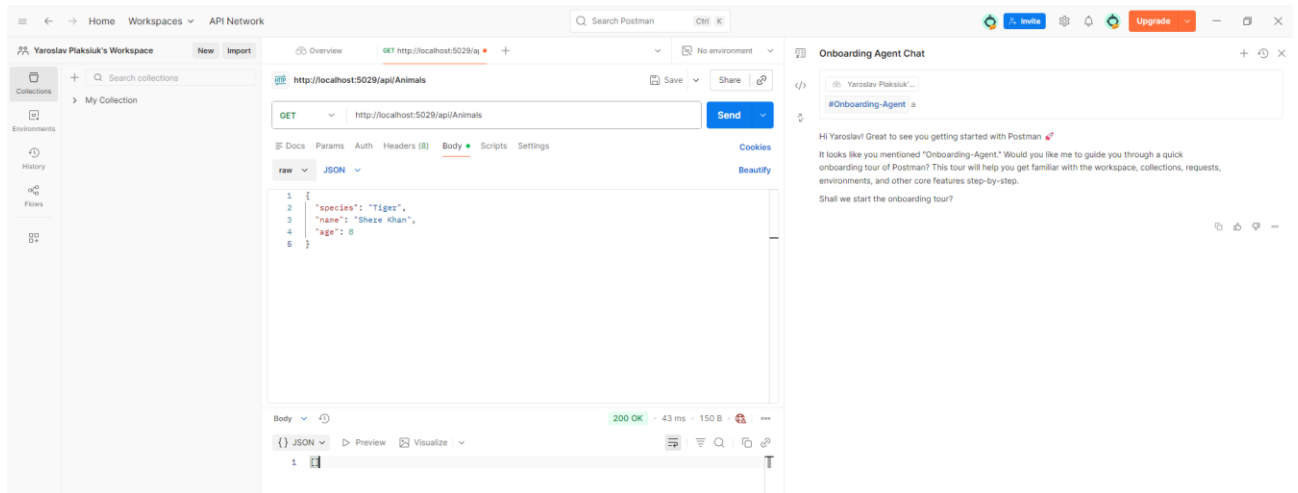


Рис. 8 – Виконання GET запиту, для Animals у Postman

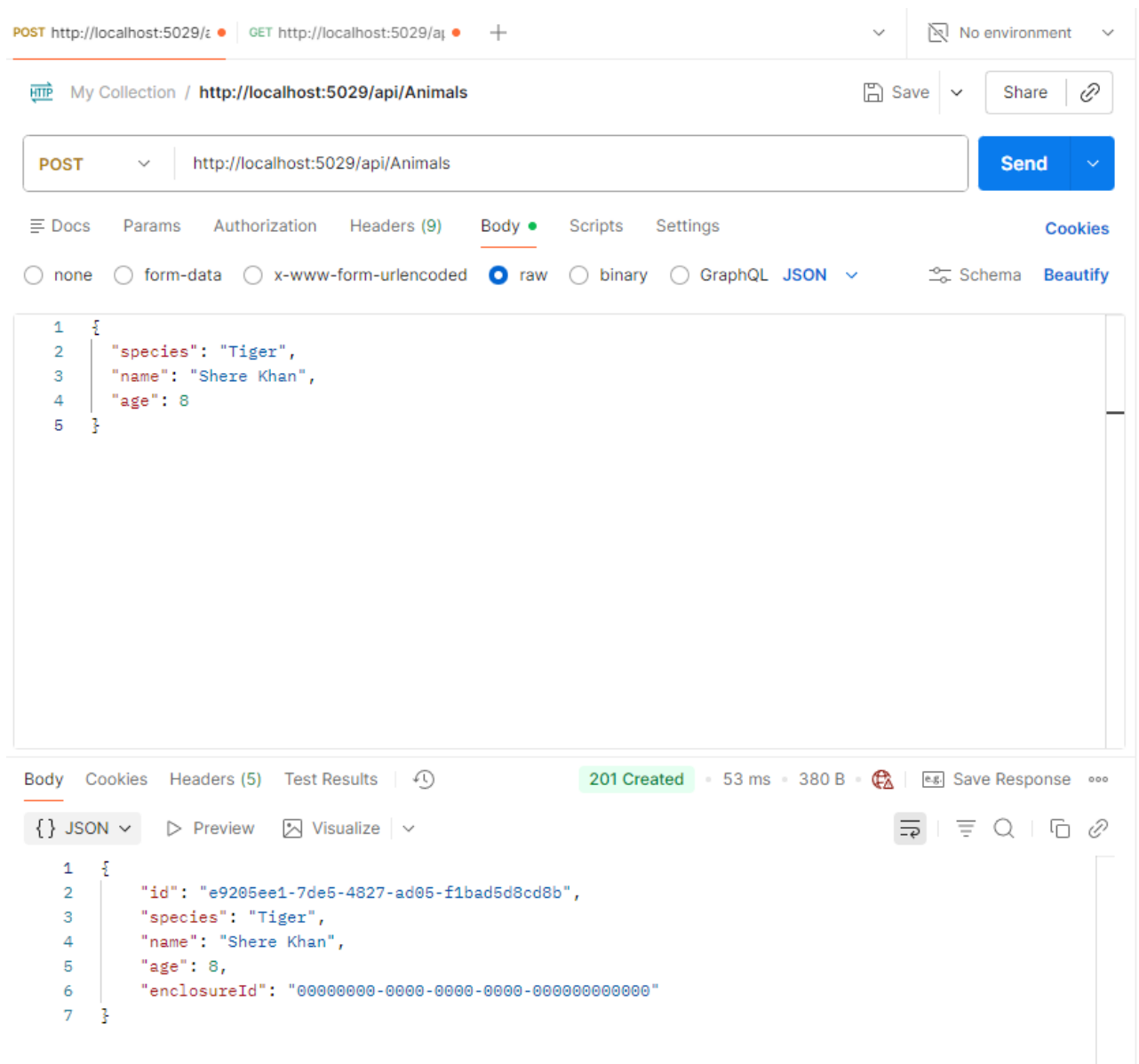


Рис. 9 – Виконання POST запиту, для Animals у Postman, для додавання

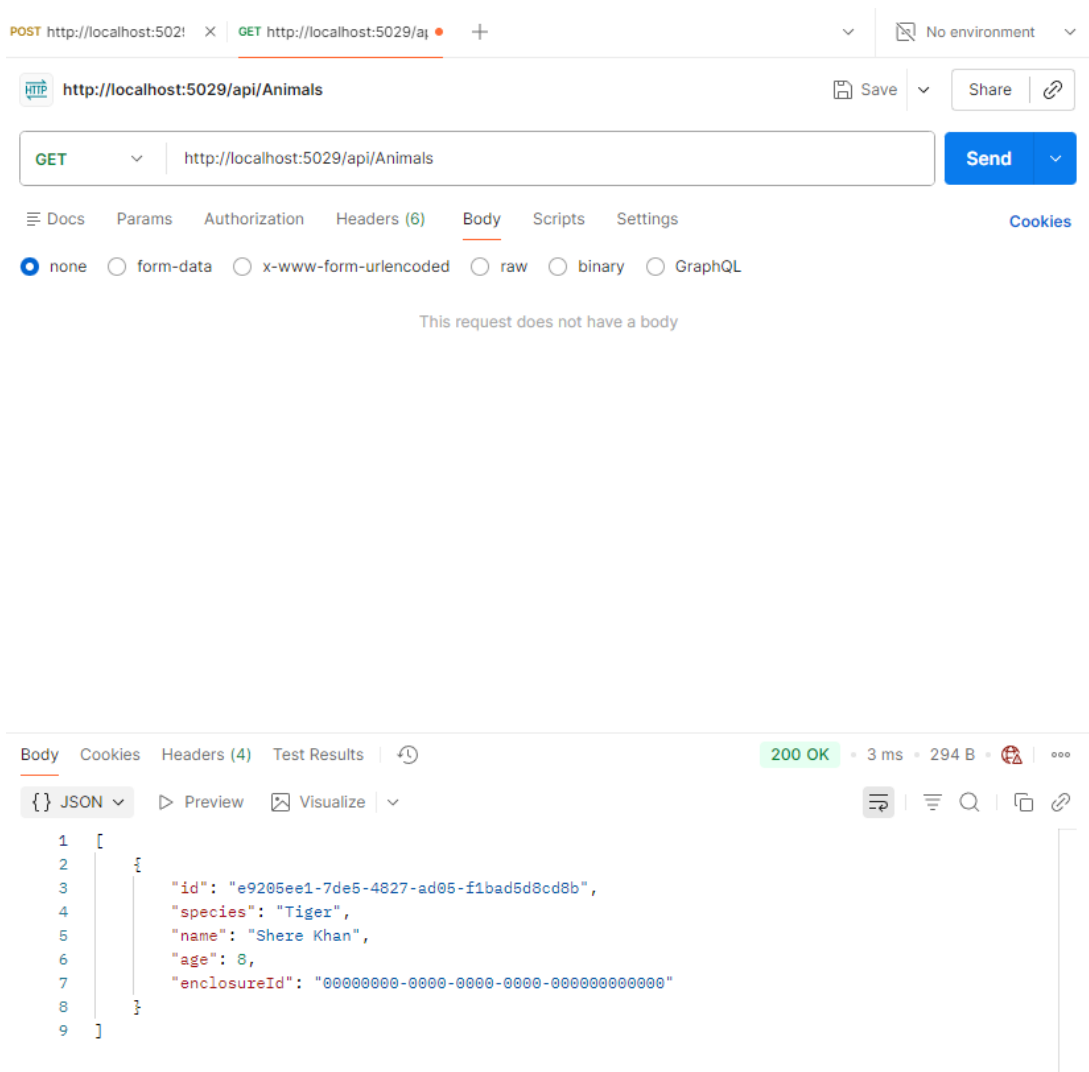


Рис. 10 – Перевірка того, що новий об'єкт додався

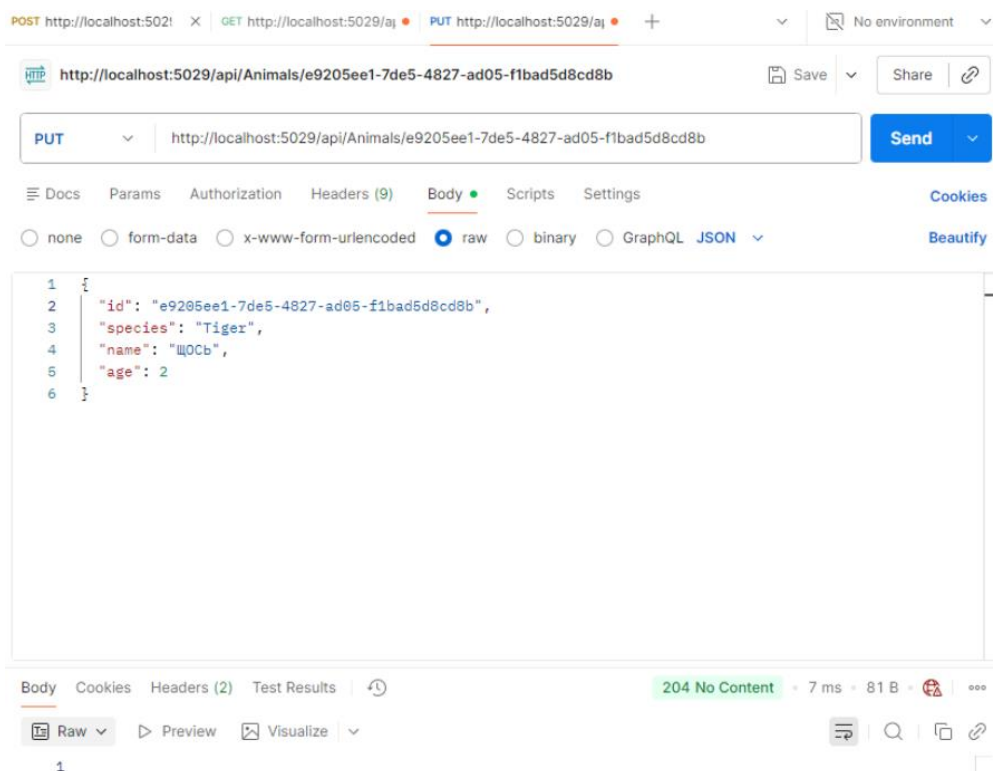


Рис. 11 – Оновлення існуючого об'єкту

POST http://localhost:5029/ GET http://localhost:5029/api/ PUT http://localhost:5029/api/ + No environment

HTTP http://localhost:5029/api/Animals Save Share

GET http://localhost:5029/api/Animals Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (4) Test Results 200 OK 2 ms 292 B

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": "e9205ee1-7de5-4827-ad05-f1bad5d8cd8b",
4     "species": "Tiger",
5     "name": "ЩОСЬ",
6     "age": 2,
7     "enclosureId": "00000000-0000-0000-0000-000000000000"
8   }
9 ]
```

Рис. 12 – Перевірка того, що об'єкт оновився

POST http://loc: GET http://localhc PUT http://localhc GET http://localhc DEL http://localhc + No environment

HTTP http://localhost:5029/api/Animals/e9205ee1-7de5-4827-ad05-f1bad5d8cd8b Save Share

DELETE http://localhost:5029/api/Animals/e9205ee1-7de5-4827-ad05-f1bad5d8cd8b Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body Cookies Headers (2) Test Results 204 No Content 7 ms 81 B

Raw Preview Visualize

1

Рис. 13 – Видалення об'єкту

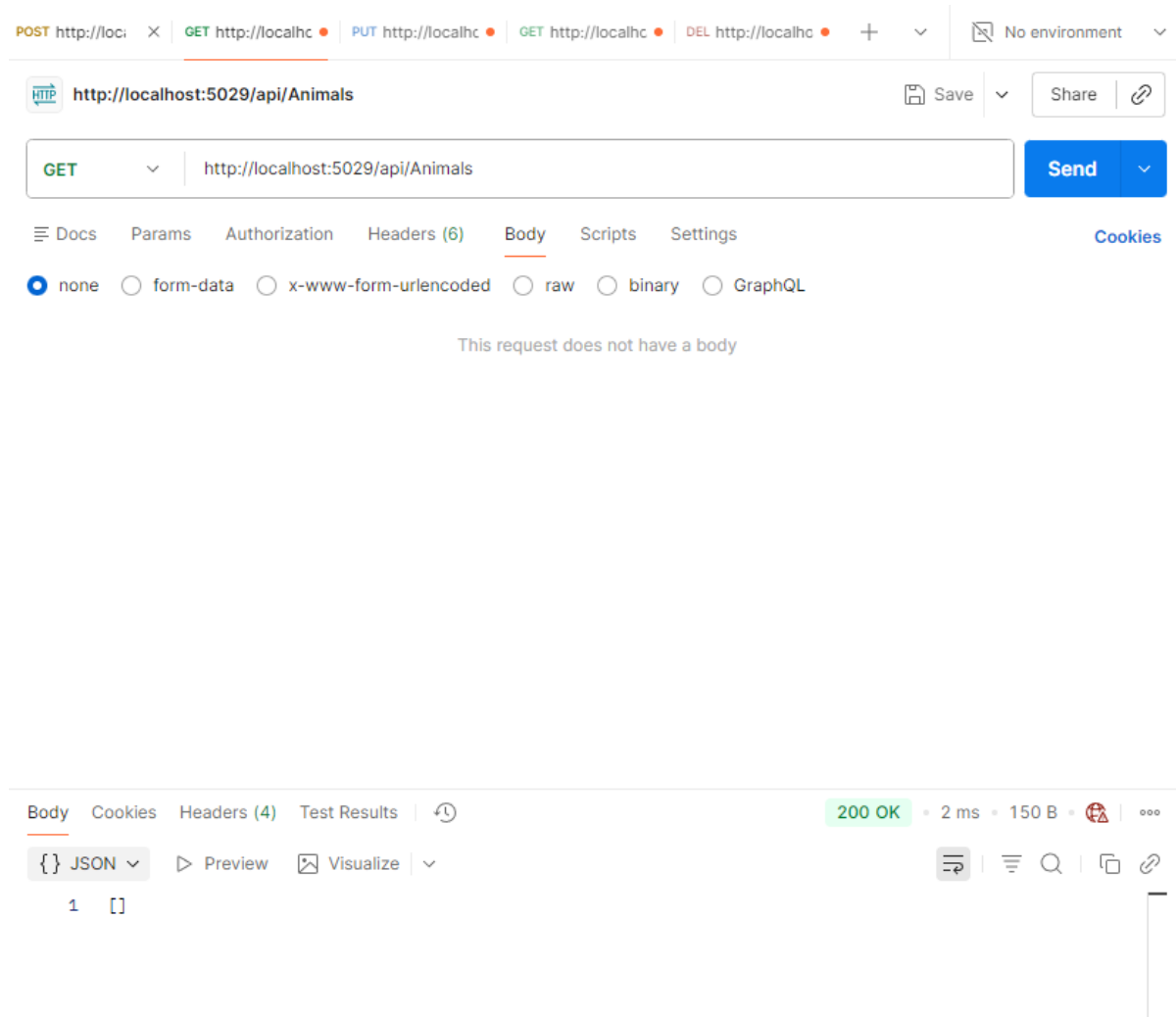


Рис. 14 – Перевірка того, що об’єкт видалився

Висновок

Під час виконання лабораторної роботи я створив у Visual Studio мій проект Zoo за шаблонами ASP.NET. Після цього були написані відповідні моделі та контролери та виконані запити у Postman.